

Industrial Functional Programming

Melinda Tóth, István Bozó



Dept. Programming Languages and Compilers
Eötvös Loránd University, Budapest, Hungary

Contents

- 1 Tracing
- 2 DBG tracer
- 3 Profiling
- 4 Code Coverage
- 5 Others

Trace BIF-s

- Low level trace event handling
 - global and local function calls
 - garbage collection, memory usage
 - process related activities, message passing
- `erlang:trace/3`
- Events arrive in messages:
`{trace, Pid, Tag, Data},`
`{trace, Pid, Tag, Data, Data2}`

Trace BIF-s

```
7> Pid = spawn(fun pingpong:proc/0).
8> erlang:trace(Pid, true, [send, 'receive']).
1
9> flush().
ok
10> pingpong:ping(Pid).
<0.60.0> is alive!
pong
11> flush().
Shell got {trace,<0.60.0>,'receive',{ping,<0.67.0>}}
Shell got {trace,<0.60.0>,send,pong,<0.67.0>}
```

Trace BIF-s

- `erlang:trace(PidSpec, Bool, TraceFlags)`
- `PidSpec`: `Pid`, `all`, `new`, `existing`
- `erlang:trace(self(), ...)` – `badarg`
- `Bool`: turn tracer on or off
- Trace flags:
 `all`, `send`, `'receive'`, `call`, `return_to`,
 `procs`, `running`, `exiting`,
 `garbage_collection`, `set_on_spawn`,
 `{tracer, Pid}`
<http://erlang.org/doc/man/erlang.html#trace-3>

Trace Patterns to Trace Calls

- `erlang:trace_pattern(MFA, MatchSpec, Flags)`
- **Only calls of traced processes:**
`erlang:trace(PidSpec, Bool, [calls, return_to])`
- **MFA:**
`{M,F,A} | {M,'_','_'} | {'_','_','_'} | on_load`
- **MatchSpec:**
`true | false | pause | restart | MatchSpecList`
- **Trace flags:** `global, local, call_time`
http://erlang.org/doc/man/erlang.html#trace_pattern-3

Match Specifications

- http://erlang.org/doc/apps/erts/match_spec.html
- {MHead, MCond, MBody}, {[], [], []}
- Head, pattern match on function parameters:
`['$1', '$2'], ['$1', '$1'], ['_']`
- Cond: `true`, `false`, `[]` or a guard like nested condition:
`[{'<', '$1', 4}, {'>', {hd, '$2'}, '$3']}`
- Body, list of actions to perform (sending msg, return trace information, enable/disable flags, tracing, logging):
`{return_trace}, {enable(disable)_trace, Flag}, {display, term()}, {message, term()}`

Trace Patterns to Trace Message Passing

- `erlang:trace_pattern(send | 'receive', MSpec)`
- No pause, restart in MSpec.

dbg

- "The Text Based Trace Facility"

- `dbg:h()`

- **Workflow:**

`dbg:tracer(), dbg:p(...), ..., dbg:stop()`

- **Clearing the flags:** `dbg:stop_clear()`

- **Tracing calls:** `dbg:tp/2`, `dbg:tpl/2`

- **Trace and evaluate a function:** `dbg:c/1,2`

- **Handling the output:**

`dbg:tracer(process, {HandlerFun, Data})`

fun2ms

- fun expression representation to generate match specifications

```
dbg:fun2ms (fun ([N,N]) when N > 3 ->  
              return_trace()  
            end) .
```

```
[[{ '$1', '$1' },  
  [{ '>', '$1', 3 }],  
  [{ return_trace }]]]
```

cprof

```
cprof:start() .
```

```
mymod:...
```

```
cprof:pause()
```

```
cprof:analyze(mymod) .
```

fprof

```
fprof:trace(start).
```

```
mymod:...
```

```
fprof:profile()
```

```
fprof:analyze({dest, mymoddata}).
```

```
fprof:trace(stop).
```

percept, percept2

http://erlang.org/documentation/doc-8.3/lib/percept-0.9/doc/html/percept_ug.html

```
percept:profile("mod.dat", {M, F A}, [procs]).
```

M:...

```
percept:analyze("mod.dat").  
percept:start_webserver(8888).
```

Cover

<http://erlang.org/doc/man/cover.html>

```
cover:start().
```

```
cover:compile(Mods)
```

```
mymod:...
```

```
cover:analyze(mymod).
```

Other tools

- xref
- debugger
- observer

Miscellaneous

- sys
- gen_tcp, gen_udp, gen_sctp
- gen_fsm, gen_statem
- os
- re
- erl_tracer
- error_logger
- <http://erlang.org/doc/man/>

Static analyzer tool

- RefactorErl :) – <https://plc.inf.elte.hu/erlang/>
- erl_tidy
- Typer, Dialyzer
- Wrangler
- xref
- elvis